TECHNICAL AUDIT REPORT

Dashboard API

Issues, Fixes, and How to Make the Response Faster

Endpoint: GET /api/superadmin/dashboard

File: app/controllers/superadmin/dashboard.controller.js — 2,064 lines, 292 awaits

Stack: Node 18 · Express 4 · Sequelize 6 · MySQL (remote host)

Audit date: 29 July 2026

Prepared for: Rahul Kumar

Headline: one dashboard request issues roughly 134 database queries, almost all of them sequential, against a MySQL server on a remote host — and none of the indexes that exist can serve them.

Method: Static source analysis of the controller, its helper library, the Sequelize models, the migration history and the server bootstrap. No production traffic, slow-query log or EXPLAIN output was available; query counts are counted from source, timing figures are analytical estimates and are labelled as such.

1. Executive Summary

This dashboard is slow for four reasons that multiply together. Any one of them alone would be survivable; combined, they put the endpoint into the multi-second range and keep it there regardless of hardware.

1. Every index on the tables it reads is unusable

One migration (`20221201092123-add_index_in_tables.js`) added composite indexes to 30 tables, and every one of them leads with the `id` column. Because `id` is the primary key and unique, a composite index led by it can only ever serve lookups by id — MySQL's leftmost-prefix rule means no other query can use it. The dashboard never filters by id. It filters by `role_id`, `own`, `parent_id`, `type`, `is_approved`, `is_assigned` and `is_approval` — and of those, only `parent_id` appears in any index at all, in a position that cannot be reached. Every dashboard query is a full table scan.

2. The aggregation is done in JavaScript instead of SQL

`getTotalStockByUser` selects every matching stock row and adds up the quantities in a `for` loop. It is called seven times on the super-admin path. `getTotalStockPriceByUser` is worse: it loads a five-level object graph per stock row — stock → stock materials → material → material price → purity prices, plus product, category, sub-category and tax — and computes prices row by row in Node. It is also called seven times. `getOwnUserSaleProducts` loads every sale with an eight-level include tree and sums it in JavaScript.

None of these needs to transfer a single row. They are SUM() queries wearing a disguise.

3. Roughly 134 queries, run one after another

The super-admin path issues about 134 statements per request, and they are almost entirely sequential — each await blocks the next, even where the results are completely independent. The twelve-month chart alone is a while loop that fires three queries per iteration: 36 sequential round-trips to draw one graph that a single GROUP BY would answer.

4. The database is on a remote host

`config/config.js` points every environment at an external IP address rather than a local socket. So each of those 134 queries pays a full network round-trip. At a modest 10 ms that is 1.3 seconds of pure waiting; at 25 ms — typical for a cross-datacentre hop — it is 3.4 seconds, before MySQL has done any work. This is why the endpoint feels slow even when the tables are small: the cost is dominated by round-trip count, not by data volume.

1.1 The measurement problem

You currently have no way to see any of this. Sequelize is configured with `logging: false`, so no SQL is ever printed. And `app/middlewares/requestLogger.js` — which looks like request timing — computes the elapsed time on the line immediately after starting the timer, before the handler has run. It always reports approximately zero milliseconds. The file write is commented out, so nothing is recorded anywhere regardless. Fixing that middleware is the cheapest change in this report and it should be done first, because it is what will tell you whether everything else worked.

1.2 What the fix is worth

The four causes have very different cost-to-benefit profiles. In rough order of return:

Change	Effort	Est. saving	Why
Add usable indexes (Section 6)	2 hours	Very large	Turns 134 table scans into index seeks
Push SUM into SQL (Q2, Q3, Q4)	2 days	Very large	Stops transferring rows to count them
GROUP BY for the month chart (Q5)	3 hours	36 → 3 queries	Removes 33 sequential round-trips
Parallelise independent awaits (P1)	1 day	Large	Wall time becomes the slowest, not the sum
Cache the payload for 60s (P3)	3 hours	Large	Repeat loads cost nothing
Fix the request timer (O1)	20 min	—	Makes all of the above verifiable

Sections 3–5 give each issue with its exact location, why it costs what it costs, and the replacement code. Section 6 is a ready-to-run index script. Section 7 is a sequenced plan with a target response-time budget.

2. Findings at a Glance

ID	Finding	Severity	Effort	Class
Q-1	Every composite index leads with id — none is usable	Critical	Low	Query
Q-2	getTotalStockByUser sums in JavaScript, not SQL (7 calls)	Critical	Medium	Query
Q-3	getTotalStockPriceByUser loads a 5-level graph per row (7 calls)	Critical	High	Query
Q-4	getOwnUserSaleProducts loads every sale with an 8-level include	Critical	High	Query
Q-5	12-iteration month loop = 36 sequential queries	Critical	Low	Query
Q-6	IDs round-tripped into JS and passed back as huge IN() lists	High	Medium	Query
Q-7	getSuperAdminId does SELECT * and is never cached	High	Low	Query
Q-8	No LIMIT on any helper query	Medium	Medium	Query
P-1	~134 queries per request, almost all sequential	Critical	Medium	Performance
P-2	Remote database — every query pays internet latency	Critical	Medium	Performance
P-3	No caching of any kind	High	Low	Performance
P-4	A single 1,090-line function serves all five roles	High	High	Maintainability
P-5	95 console.log calls on the hot path	Medium	Low	Performance
P-6	No compression; 50 MB JSON body limit	Medium	Low	Performance
P-7	indexNew — 731 lines of dead code, not routed	Medium	Low	Maintainability
P-8	Pool max 20, acquire 60 s — one request holds a connection for seconds	High	Low	Scalability
O-1	requestLogger always reports ~0 ms and writes nothing	High	Low	Observability
O-2	Sequelize logging: false — no SQL visibility at all	High	Low	Observability
S-1	Database credentials hardcoded as fallbacks in config/config.js	High	Low	Security

Severity reflects impact on the dashboard's response time and stability. Effort is engineering time for a developer familiar with this codebase.

3. Where the 134 Queries Come From

This is the super-admin execution path through exports.index. Counts are from source: direct model calls counted in the controller, helper calls expanded by reading each helper in app/library/common.js.

Source	Calls	Queries	Note
Direct model calls, lines 70–345	27	27	count / sum / findAll on users, sales, purchases
getTotalStockByUser	7	7	findAll then sum in JS (Q-2)
getTotalStockPriceByUser	7	14	Parent + separate:true child query each (Q-3)
avlStockUserIdsNew	2	12	6 queries each: superadmin id ×2, own users, managers, admin distributors, SEs
getOwnUserSaleProducts	2	18	Calls avlStockUserIdsNew (6) + sales + 2 separate includes (Q-4)
Common section, lines 827–911	~20	~20	Purchase sum, wallet balance, chart id resolution
Month chart loop, lines 912–1053	12 iters	36	3 queries per iteration (Q-5)
Total (super-admin path)	—	≈ 134	Essentially all sequential

3.1 What sequential means in wall-clock terms

Because the awaits are chained rather than batched, the request's duration is the sum of every round-trip, not the maximum. The table below models that against round-trip latency to the remote MySQL host. These are analytical estimates: they assume MySQL itself answers instantly, which after Q-1 it will not.

Round-trip latency	Network wait alone	Plus scan time (Q-1)	Scenario
2 ms (same host)	~0.3 s	—	Not your deployment
10 ms (same region)	~1.3 s	3–5 s	Optimistic
25 ms (cross-region)	~3.4 s	6–12 s	Likely current
50 ms (poor link)	~6.7 s	12–25 s	Under load or contention

The key insight: at 134 sequential queries you cannot fix this with a bigger database server. The cost is round-trip count. Cutting the query count is the only lever that works, and it is why the fixes in Section 4 are ordered the way they are.

4. Query Issues and Fixes

Q-1 Every composite index leads with id, so none of them is usable [Critical]

Location `migrations/20221201092123-add_index_in_tables.js — the only migration that adds indexes`

Problem That migration indexes 30 tables. Every index puts id first:

```
addIndex('users',     ['id','parent_id','state_id','district_id','country_id'], ...)
addIndex('sales',     ['id','user_id','order_id','sale_by','invoice_date'],     ...)
addIndex('stocks',    ['id','product_id','size_id','purchase_id','user_id','total_weight'], ...)
addIndex('purchases', ['id','supplier_id','user_id','sale_id','invoice_date','is_assigned'], ...)
```

MySQL can only use a composite index from the left. Because id is the primary key and unique, matching on it identifies exactly one row — so the remaining columns of the index can never contribute. And a query that does not filter on id cannot enter the index at all.

What the dashboard actually filters on None of it is reachable:

Table	Dashboard predicate	Index status
users	role_id = ? AND own = ? AND parent_id IN (...)	role_id, own indexed nowhere
sales	sale_by IN (...) AND is_approved <> 2 AND is_assigned = 0 AND is_approval = 0	3 boolean flags indexed nowhere
sales	... AND invoice_date BETWEEN ? AND ?	in an index, unreachable position
stocks	type = ? AND user_id IN (...)	type indexed nowhere
purchases	user_id = ? AND is_approved <> 2 AND is_assigned = 0	is_approved, is_approval missing
orders	order_from = ? AND createdAt BETWEEN ? AND ?	neither column indexed

Consequence Every one of the ~134 queries is a full table scan. The users table is scanned dozens of times per request. This is the single highest-return finding in the report and the cheapest to fix — it is pure DDL, no application code changes.

Fix Add narrow indexes matching the real predicates. Section 6 has the full script. Verify each with EXPLAIN — you want type: ref or range, not ALL.

```
-- The four that matter most
CREATE INDEX ix_users_role_own_parent ON users (role_id, own, parent_id);
CREATE INDEX ix_sales_flags_saleby    ON sales (is_approval, is_assigned, is_approved, sale_by,
invoice_date);
CREATE INDEX ix_stocks_type_user      ON stocks (type, user_id);
CREATE INDEX ix_purchases_user_flags  ON purchases (user_id, is_approval, is_assigned, is_approved);
```

Q-2 getTotalStockByUser sums in JavaScript instead of SQL [Critical]

Location `app/library/common.js — getTotalStockByUser; called 7× on the super-admin path`

```
let stocks = await StockModel.findAll({ where: conditions });
for (let i = 0; i < stocks.length; i++) {
  qty += stocks[i].quantity ? parseInt(stocks[i].quantity) : 1;
}
return qty;
```

Problem Every matching stock row is selected with all columns, transferred over the network, instantiated as a Sequelize model object, and then added up in Node. For a total. With no index on (type, user_id) this is a full scan of stocks, seven times per request, each time with a different user-id set.

The quantity ? x : 1 detail The fallback treats a null quantity as 1. That behaviour has to be preserved in SQL or the numbers will change — use COALESCE, and confirm with the business whether null-means-one is actually intended.

Fix One aggregate query, no rows transferred.

```
const getTotalStockByUser = async (userId, type = 'product') => {
  const ids = Array.isArray(userId) ? userId : [userId];
  if (!ids.length) return 0;

  const [row] = await dbSequelize.query(
    `SELECT COALESCE(SUM(COALESCE(quantity, 1)), 0) AS qty
       FROM stocks
      WHERE type = :type AND user_id IN (:ids)`,
    { replacements: { type, ids }, type: QueryTypes.SELECT }
  );
  return Number(row.qty) || 0;
};
```

Better still The seven calls differ only by user-id set and type. Replace them with one grouped query that returns every bucket at once, then read the results from a Map — seven queries becomes one.

```
SELECT user_id, type, SUM(COALESCE(quantity,1)) AS qty
  FROM stocks
 WHERE user_id IN (:allIds)
 GROUP BY user_id, type;
```

Q-3 getTotalStockPriceByUser loads a five-level object graph per stock row [Critical]

Location `app/library/common.js – getTotalStockPriceByUser; called 7× on the super-admin path`

Problem The include tree fetched for every stock row is:

```
stock
  └─ stockMaterials            (separate: true → its own query)
       ├─ material
       │    └─ material_price
       │         └─ materialPricePurities
       ├─ unit
       └─ purity
  └─ product                   (required: true)
       ├─ category
       ├─ sub_category
       └─ tax
```

Every row of that graph crosses the network and becomes a Sequelize instance. calculateProductPrice then runs per stock row in JavaScript, and a lodash findIndex runs per row against the growing categories array — which makes the category rollup $O(n^2)$ in the number of distinct categories.

Cost Two queries per call (parent plus the separate child), but each returns the full graph. Seven calls per request. This is the most expensive helper in the dashboard by data volume, and its result is usually a single number — a total stock price.

Fix — staged This one cannot be rewritten in an afternoon, because the pricing arithmetic in calculateProductPrice encodes real business rules. Three steps, in order:

- **Step 1 — project.** Add an attributes list to every level of the include tree so only the columns the price calculation reads are fetched. This is safe, mechanical, and typically cuts the transferred bytes by 80% on its own.
- **Step 2 — deduplicate.** Material prices and purities are shared reference data. Load them once per request into a Map instead of re-joining them for every stock row.
- **Step 3 — precompute.** Stock valuation changes only when stock or prices change. Store a computed unit_value on the stock row, maintained on write, and the dashboard becomes SUM(unit_value * quantity) — one query, no graph, no JavaScript arithmetic.

Interim Until step 3 lands, cache the result. Stock valuation for a whole role tree does not need to be accurate to the second on a dashboard tile.

Q-4 getOwnUserSaleProducts loads every sale with an eight-level include tree [Critical]

Location `app/library/common.js — getOwnUserSaleProducts; called 2× on the super-admin path`

```
sale
  └─ saleProducts              (separate: true)
       ├─ product — category, sub_category
       ├─ size
       └─ saleMaterials        (separate: true) — material, purity, unit
  └─ saleBy (full user row)
```

Problem There is no date filter and no LIMIT. The query returns every sale ever recorded for every user in the tree, with the entire product and material graph attached, and then sums total_amount and counts products in a JavaScript loop. The saleBy include pulls the complete user row — including the password hash — for every sale.

Also It calls avlStockUserIdsNew internally, which is six more queries, and the dashboard calls it twice — so the six-query id resolution runs twice for the same answer.

Fix The dashboard needs two numbers from this: total amount and total product count. Both are aggregates.

```
SELECT COALESCE(SUM(sp.total), 0)  AS total_amount,
       COUNT(sp.id)                AS total_product
  FROM sales s
  JOIN sale_products sp ON sp.sale_id = s.id
 WHERE s.sale_by IN (:userIds)
   AND s.is_approved <> 2
   AND s.is_assigned = 0
   AND s.is_approval = 0;

-- category rollup, same shape, one more query instead of a JS loop
SELECT p.category_id, c.name AS category_name,
       SUM(sp.total) AS total_amount, COUNT(sp.id) AS quantity
  FROM sales s
  JOIN sale_products sp ON sp.sale_id = s.id
  JOIN products p       ON p.id = sp.product_id
  LEFT JOIN categories c ON c.id = p.category_id
 WHERE s.sale_by IN (:userIds) AND s.is_approved <> 2
   AND s.is_assigned = 0 AND s.is_approval = 0
 GROUP BY p.category_id, c.name;
```

Caution The existing JS loop has special-case logic for material-type products and for rows with an empty certificate_no. Read it carefully before porting — the SQL above is the shape, not a drop-in replacement, and the totals must be reconciled against the current output before you switch over.

Q-5 The twelve-month chart is 36 sequential queries [Critical]

Location `dashboard.controller.js:912–1053 — while (month < 13)`

```
let month = 1;
while (month < 13) {
  let month_range = getMonthDateRange(moment().format('YYYY'), month);
  ...
  currMonthCustomer = await UserModel.count({ where: { role_id, createdAt: {...} } });
  currMonthOrder    = await OrderModel.sum('total_amount', { where: {...} });
  currMonthSales    = await saleModel.sum('total_payable', { where: {...} });
  month++;
}
```

Problem Three queries per iteration, twelve iterations, every one awaited before the next begins. 36 round-trips to produce three arrays of twelve numbers. Each of those queries is also a full scan today (Q-1).

Fix Three GROUP BY queries — one per series — each returning all twelve months at once. Run them in parallel.

```
const year = moment().format('YYYY');

const [customers, orders, sales] = await Promise.all([
  dbSequelize.query(
```

```
    `SELECT MONTH(createdAt) AS m, COUNT(*) AS v
       FROM users
      WHERE role_id = :roleId AND YEAR(createdAt) = :year
      GROUP BY MONTH(createdAt)`,
    { replacements: { roleId: customerRoleId, year }, type: QueryTypes.SELECT }),

  dbSequelize.query(
    `SELECT MONTH(createdAt) AS m, COALESCE(SUM(total_amount),0) AS v
       FROM orders
      WHERE order_from = 'front_website' AND YEAR(createdAt) = :year
      GROUP BY MONTH(createdAt)`,
    { replacements: { year }, type: QueryTypes.SELECT }),

  dbSequelize.query(
    `SELECT MONTH(invoice_date) AS m, COALESCE(SUM(total_payable),0) AS v
       FROM sales
      WHERE sale_by IN (:ids) AND is_approved <> 2
        AND is_assigned = 0 AND is_approval = 0
        AND YEAR(invoice_date) = :year
      GROUP BY MONTH(invoice_date)`,
    { replacements: { ids: avl_stockUser_ids, year }, type: QueryTypes.SELECT }),
]);

// Fill the 12 slots — GROUP BY omits months with no rows
const toSeries = (rows) => {
  const out = Array(12).fill(0);
  rows.forEach((r) => { out[Number(r.m) - 1] = Number(r.v) || 0; });
  return out;
};
const customerMonthwise = toSeries(customers);
```

Note the gap-filling. GROUP BY returns no row for a month with no data, whereas the current loop pushes a 0. toSeries preserves that, so the chart keeps its twelve points.

Index note: YEAR(createdAt) wraps the column in a function, which prevents index use. If these tables are large, switch to a range predicate instead — createdAt >= '2026-01-01' AND createdAt < '2027-01-01' — which an index on createdAt can serve.

Q-6 IDs round-tripped into JavaScript and passed back as large IN() lists [High]

Location `avlStockUserIdsNew and the branch code throughout dashboard.controller.js:70–345`

Problem The controller repeatedly does: query for a set of user ids, pull them into an array with arrayColumn, then send that array back to MySQL as an IN() list for the next query. avlStockUserIdsNew does this six times to walk the super-admin → admin → distributor → sales-executive tree.

Two costs. Every hop is an extra round-trip. And the IN() lists grow with the organisation — on a large tenant these become multi-kilobyte statements that MySQL cannot cache a plan for, and which will eventually hit max_allowed_packet.

Fix Express the hierarchy as a join or a subquery so MySQL resolves it in one pass. A recursive CTE handles arbitrary depth (MySQL 8+):

```
WITH RECURSIVE tree AS (
  SELECT id, role_id, own FROM users WHERE id = :rootId
  UNION ALL
  SELECT u.id, u.role_id, u.own
    FROM users u JOIN tree t ON u.parent_id = t.id
)
SELECT id FROM tree WHERE own = 1;
```

If you are on MySQL 5.7 Recursive CTEs are unavailable. Keep the id-array approach but cache the resolved tree per user for the request's lifetime — avlStockUserIdsNew is called twice with identical arguments on the super-admin path, so memoising it alone removes six queries.

Q-7 getSuperAdminId does SELECT * and is never cached [High]

Location `app/library/common.js – getSuperAdminId`

```
const getSuperAdminId = async () => {
  let user = await UserModel.findOne({ where: { role_id: getRoleId('superadmin') } });
  return user.id;
};
```

Problem Three issues in four lines. There is no attributes projection, so the entire user row — password hash, addresses, images — is fetched to read one integer. There is no index on role_id (Q-1), so it is a full scan. And it is called four or more times per dashboard request: twice inside each avlStockUserIdsNew call, which itself runs twice.

Also user.id will throw if no super admin exists. There is no null guard.

Fix Project, guard, and cache at module scope. The super admin's id does not change at runtime.

```
let _superAdminId = null;
const getSuperAdminId = async () => {
  if (_superAdminId) return _superAdminId;
  const user = await UserModel.findOne({
    attributes: ['id'],
    where: { role_id: getRoleId('superadmin') },
    order: [['id', 'ASC']],
  });
  if (!user) throw new Error('No superadmin user found');
  _superAdminId = user.id;
  return _superAdminId;
};
```

Note: getRoleId is already a hardcoded switch returning constants, so role ids cost nothing. Applying the same treatment to the super-admin id closes the gap.

Q-8 No LIMIT on any helper query [Medium]

Problem getTotalStockByUser, getTotalStockPriceByUser and getOwnUserSaleProducts all select unbounded result sets. Their cost scales with total historical data volume, so the dashboard gets slower every month regardless of usage — which is the pattern that makes performance regressions look like they came from nowhere.

Fix Once Q-2 through Q-4 are aggregates this disappears, because aggregates return one row. Until then, add a date bound wherever the business logic allows it — most dashboard figures do not need sales from four years ago.

5. Performance, Scalability and Observability

P-1 ~134 queries per request, almost all sequential [Critical]

Problem The controller is written as a long chain of awaits. Most of them are independent — the admin count does not depend on the distributor stock total — but each still waits for the previous to return.

```
totalCustomer     = await UserModel.count({ ... });      // independent
totalStock        = await getTotalStockByUser(userID);   // independent
totalStockPrice   = await getTotalStockPriceByUser(...); // independent
materialTotalStock= await getTotalStockByUser(userID, 'material');
totalSupplier     = await UserModel.count({ ... });      // independent
saleDueAmount     = await saleModel.sum('due_amount', { ... });
// six round-trips that could have been one Promise.all
```

Fix Group the independent calls. The pattern is mechanical: identify which awaits consume a previous result, batch everything else.

```
const [totalCustomer, totalStock, totalStockPrice,
       materialTotalStock, totalSupplier, saleDueAmount] = await Promise.all([
  UserModel.count({ where: { role_id: customerRoleId } }),
  getTotalStockByUser(userID),
  getTotalStockPriceByUser(null, userID),
  getTotalStockByUser(userID, 'material'),
  UserModel.count({ where: { role_id: supplierRoleId, parent_id: userID } }),
  saleModel.sum('due_amount', { where: { sale_by: userID, ... } }),
]);
```

One caution: Promise.all fires everything at once, and the Sequelize pool caps at 20 (P-8). Batch in groups of 8–10 rather than firing all 134 simultaneously, or you will simply move the queue from the event loop into the pool.

P-2 The database is on a remote host [Critical]

Location `config/config.js — all three environments point at an external IP`

Problem Development, test and production all target the same external address on different ports. Every query is an internet round-trip. At 134 sequential queries, latency dominates everything else — the endpoint would still take seconds against an empty database.

Fix Two independent directions, both worth doing:

- Reduce the query count. This is the durable fix and it is what Section 4 is for — it makes the endpoint fast regardless of where the database lives.
- Reduce the latency. Move the application and the database into the same region or private network. This multiplies the benefit of everything else.

Measure it first: run SELECT 1 in a loop from the app host and record the median round-trip. That single number tells you how much of your response time is physics rather than code, and it takes two minutes to obtain.

P-3 No caching of any kind [High]

Problem Nothing is cached — not the role tree, not the stock valuations, not the month chart, not the response. Two admins opening the dashboard run the full 134-query path twice. One admin refreshing runs it again.

Fix Three layers, cheapest first.

- **Response cache.** Key on user id and role, 60-second TTL. A dashboard figure that is a minute stale is invisible to the user and removes essentially all repeat load.
- **Reference cache.** The super-admin id (Q-7) and the resolved role tree (Q-6) change rarely. Cache at module scope with explicit invalidation on user create/update.

- **Precomputed aggregates.** For the expensive valuations (Q-3), a scheduled job writing to a summary table turns the dashboard into a handful of primary-key reads. This is the endgame, not the starting point.

```
// app/library/dashboardCache.js
const cache = new Map();
const TTL = 60 * 1000;

const remember = async (key, build) => {
  const hit = cache.get(key);
  if (hit && Date.now() - hit.at < TTL) return hit.value;
  const value = await build();
  cache.set(key, { value, at: Date.now() });
  return value;
};
const invalidate = (prefix) => {
  for (const k of cache.keys()) if (k.startsWith(prefix)) cache.delete(k);
};
module.exports = { remember, invalidate };
```

P-4 A single 1,090-line function serves all five roles [High]

Location `dashboard.controller.js:63–1153 — exports.index`

Problem One function handles super admin, admin, distributor, sales executive and manager through nested if/else branches, declaring around 50 mutable let variables at the top and assigning into them from every branch. Several are assigned without declaration — the block at lines 85–126 uses comma-chained assignment, creating implicit globals in non-strict mode.

Why it matters for performance Not directly — but it is why the sequential-await problem is hard to fix safely. You cannot confidently batch calls when any branch might reassign a shared variable. Splitting the function per role is a prerequisite for the P-1 work, not a nice-to-have afterwards.

Fix Extract one builder per role — buildSuperAdminDashboard(req), buildAdminDashboard(req) and so on — each returning its own object. The route handler picks the builder and serialises the result. Do this before batching, not after.

P-5 95 console.log calls on the hot path [Medium]

Problem 70 in app/library/common.js and 25 in dashboard.controller.js. Several print entire result sets — getTotalStockPriceByUser logs the whole categories array, getOwnUserSaleProducts logs the sales length and the items length. console.log to a pipe is synchronous, so on a single-threaded Node process each one blocks the event loop, and serialising a large array to print it is itself expensive.

Fix Replace with a levelled logger that compiles out below the configured level. Never log a full result set — log its length.

P-6 No compression; 50 MB JSON body limit [Medium]

Location `server.js — bodyParser.json({ limit: '50mb' }); no compression middleware in package.json`

Problem The dashboard response has more than 40 keys including four twelve-element chart arrays and category breakdowns, and it goes over the wire uncompressed. Separately, a 50 MB JSON limit means a single large body can block the event loop for seconds during parse — JSON.parse is synchronous.

Fix Add compression, and reduce the body limit to what uploads actually need. If image uploads go through multer they do not use the JSON parser at all, so the 50 MB is likely vestigial.

```
npm install compression
app.use(require('compression')());   // server.js, before the routes
```

P-7 indexNew — 731 lines of dead code [Medium]

Location `dashboard.controller.js:1155–1886 — exports.indexNew, not referenced by any route`

Problem A second full dashboard implementation exists and is not wired to anything. It contains 113 awaits and its own commented-out month loop. It is a third of the file. Anyone optimising the dashboard has to work out which of the two is live before they can start, and a fix applied to the wrong one is silently ineffective.

Fix Delete it, or if it represents work in progress, move it to a branch. Git remembers; the file does not need to.

P-8 Pool max 20, acquire timeout 60 s [High]

Location `config/config.js — pool: { max: 20, min: 0, acquire: 60000, idle: 10000 }`

Problem A request issuing 134 sequential queries holds its pooled connection for the entire duration — seconds. With max 20, roughly 20 concurrent dashboard loads exhaust the pool. The 60-second acquire timeout then means the 21st request waits up to a minute before failing, which presents to the user as a hang rather than an error.

Also min: 0 means the pool drains when idle, so the first request after a quiet period pays full connection-establishment cost — to a remote host (P-2), that is a noticeable extra delay on the first dashboard load of the morning.

Fix Set min to 5 so warm connections survive idle periods, and reduce acquire to 10000 so contention fails fast and loudly. Do not raise max until the query count is down — that converts an application queue into database load.

```
pool: { max: 20, min: 5, acquire: 10000, idle: 30000 },
```

O-1 requestLogger always reports approximately zero milliseconds [High]

Location `app/middlewares/requestLogger.js`

```
const start = process.hrtime();
const durationInMilliseconds = getActualRequestDurationInMilliseconds(start);
//                             ^ computed on the very next line — always ~0
let status = res.statusCode;   // read before the handler runs — always 200

// fs.appendFile('logs/request_logs.txt', logToFile + '\n', err => { ... });
//  ^ the only persistence is commented out
next();
```

Problem The duration is measured immediately after the timer starts, before next() is called — so it reports the cost of the two lines between them, not the request. The status code is read before the handler has set it. And the file write is commented out, so nothing is persisted anywhere. The middleware looks like request timing and measures nothing.

Why this is the first thing to fix You cannot verify any change in this report without it. It is twenty minutes of work and it converts every estimate here into a measurement.

Fix Move the measurement into the finish event.

```
const demoLogger = (req, res, next) => {
  const start = process.hrtime.bigint();
  res.on('finish', () => {
    const ms = Number(process.hrtime.bigint() - start) / 1e6;
    const line = `${req.method} ${req.originalUrl} ${res.statusCode} ${ms.toFixed(1)}ms`;
    if (ms > 1000) console.warn(`[SLOW] ${line}`);
    else console.log(line);
  });
  next();
};
```

O-2 Sequelize logging is disabled — no SQL visibility [High]

Location `config/config.js — logging: false on all three environments`

Problem No SQL is ever printed, so there is no way to see the 134 queries, their shapes, or how long each takes. Combined with O-1, the endpoint is entirely opaque.

Fix Enable timed logging in development, and enable the MySQL slow-query log in production. Both are configuration, not code.

```
// config/config.js — development only
logging: (sql, timing) => {
  if (timing > 100) console.warn(`[SLOW SQL ${timing}ms] ${sql.slice(0, 200)}`);
},
benchmark: true,

-- MySQL, production
SET GLOBAL slow_query_log = 'ON';
SET GLOBAL long_query_time = 0.5;
```

S-1 Database credentials hardcoded as fallbacks [High]

Location `config/config.js`

Problem Each environment's config supplies a hardcoded username, password, host and port as the fallback when the environment variable is absent — including production. Anyone with repository access has working production database credentials, and a missing .env silently connects to a real database instead of failing.

Fix Remove the fallbacks and fail fast on missing configuration. Rotate the credentials afterwards, since they are in the git history.

```
const required = (name) => {
  const v = process.env[name];
  if (!v) throw new Error(`Missing required env var: ${name}`);
  return v;
};
```

Flagged here because it sits in the same file as the pool and logging settings you will be editing. It is a security matter and should be tracked separately from this report.

6. Index Script

These replace nothing — the existing id-led indexes can stay, they are simply never used for these queries. Run during a low-traffic window; MySQL 8 builds most secondary indexes online, but confirm against your version and table sizes.

Before running: check current cardinality with SHOW INDEX FROM users; and confirm the column names against your live schema. Column names below are taken from the Sequelize models, which should match, but migrations have been edited over 234 files and drift is possible.

```
-- ═══ users: the most-scanned table in the dashboard ═══════════
-- Serves: role_id = ? AND own = ? AND parent_id IN (...)
CREATE INDEX ix_users_role_own_parent ON users (role_id, own, parent_id);
-- Serves: parent_id IN (...) AND role_id = ?   (reverse lookup direction)
CREATE INDEX ix_users_parent_role     ON users (parent_id, role_id);
-- Serves: role_id = ? AND createdAt BETWEEN ?  (month chart)
CREATE INDEX ix_users_role_created    ON users (role_id, createdAt);
-- Serves: role_id = ? AND state_id = ? / district_id = ?  (admin, distributor)
CREATE INDEX ix_users_role_state      ON users (role_id, state_id);
CREATE INDEX ix_users_role_district   ON users (role_id, district_id);

-- ═══ sales: three boolean flags indexed nowhere today ═══════════
-- Flags first (low cardinality but always equality), then the join key,
-- then the date so range scans and sorts stay in the index.
CREATE INDEX ix_sales_flags_saleby
  ON sales (is_approval, is_assigned, is_approved, sale_by, invoice_date);
CREATE INDEX ix_sales_flags_user
  ON sales (is_approval, is_assigned, is_approved, user_id, invoice_date);

-- ═══ stocks: type is not indexed at all ═══════════════════════
CREATE INDEX ix_stocks_type_user ON stocks (type, user_id);
CREATE INDEX ix_stocks_user_type ON stocks (user_id, type);

-- ═══ purchases ═══════════════════════════════════════════════
CREATE INDEX ix_purchases_user_flags
  ON purchases (user_id, is_approval, is_assigned, is_approved);
CREATE INDEX ix_purchases_supplier
  ON purchases (supplier_id, is_approval, is_assigned);

-- ═══ orders: month chart ═════════════════════════════════════
CREATE INDEX ix_orders_from_created ON orders (order_from, createdAt);
CREATE INDEX ix_orders_touser_created ON orders (to_user_id, createdAt);
CREATE INDEX ix_orders_se_created   ON orders (sales_executive_id, createdAt);

-- ═══ join tables used by the aggregate rewrites in Section 4 ═══
CREATE INDEX ix_sale_products_sale ON sale_products (sale_id, product_id);
CREATE INDEX ix_stock_materials_stock ON stock_materials (stock_id);
CREATE INDEX ix_user_to_users_user  ON user_to_users (user_id, to_role_id, createdAt);

-- ═══ VERIFY — every one of these must show type ref or range ═══
EXPLAIN SELECT COUNT(*) FROM users
  WHERE role_id = 3 AND own = 1 AND parent_id IN (1,2,3);
EXPLAIN SELECT SUM(quantity) FROM stocks
  WHERE type = 'product' AND user_id IN (1,2,3);
EXPLAIN SELECT SUM(due_amount) FROM sales
  WHERE sale_by IN (1,2,3) AND is_approved <> 2
    AND is_assigned = 0 AND is_approval = 0;

-- type: ALL      = full table scan, index not used — investigate
-- type: ref/range = index seek — correct
-- Extra: Using filesort / Using temporary = sort not covered by the index
```

On the flag columns. is_approval, is_assigned and is_approved have very low cardinality, and conventional advice is not to index such columns alone. That advice does not apply here: they appear as equality predicates in every dashboard query alongside a selective column, so putting them first in a composite index lets MySQL seek straight to the qualifying range. Check the resulting cardinality with SHOW INDEX and reorder if your data distribution is unusual — for example if almost every row has is_approval = 0, lead with sale_by instead.

7. Plan and Target Budget

7.1 Day 1 — make it measurable, then index it

ID	Action	Effort	Expected effect
O-1	Fix requestLogger to measure on res.finish	20 min	Real numbers for everything below
O-2	Enable timed SQL logging in dev; slow-query log in prod	30 min	See the 134 queries
—	Measure baseline: SELECT 1 round-trip, dashboard duration	30 min	Separates latency from code
Q-1	Run the Section 6 index script	1 hour	Table scans become index seeks
—	EXPLAIN each dashboard query; confirm no type: ALL	1 hour	Verifies the indexes landed
P-8	pool min: 5, acquire: 10000	5 min	Warm connections; fast failure

Day 1 changes no application logic. Re-measure at the end of it before starting anything else — indexes alone often account for the majority of the improvement, and knowing that changes how you prioritise the rest.

7.2 Week 1 — cut the query count

ID	Action	Effort	Expected effect
Q-5	Rewrite the month loop as 3 GROUP BY queries	3 hours	36 queries → 3
Q-7	Project and cache getSuperAdminId	30 min	~4 queries → 0
Q-6	Memoise avlStockUserIdsNew per request	2 hours	12 queries → 6
Q-2	SUM() in SQL for getTotalStockByUser	4 hours	7 scans → 1 grouped query
P-5	Remove result-set logging from the hot path	2 hours	Unblocks the event loop
P-6	Add compression; reduce body limit	30 min	Smaller, faster responses

Running total after week 1: roughly 134 queries down to the mid-40s, with the remaining scans index-backed.

7.3 Week 2–3 — restructure

ID	Action	Effort	Expected effect
P-4	Split index into one builder per role	2 days	Prerequisite for safe batching
P-1	Batch independent awaits in groups of 8–10	1 day	Wall time → slowest, not sum
Q-4	Aggregate rewrite of getOwnUserSaleProducts	2 days	Stops loading every sale
P-3	60-second response cache	3 hours	Repeat loads near-free
P-7	Delete indexNew	15 min	Removes 731 lines of ambiguity

7.4 Later — the structural items

ID	Action	Effort	Expected effect
Q-3	Precomputed stock valuation column or summary table	1 week	Removes the heaviest helper entirely
P-2	Co-locate the application and database	Infra	Multiplies every other gain
S-1	Remove hardcoded credentials; rotate	3 hours	Security — track separately
—	Regression tests pinning the dashboard's numbers	3 days	Aggregate rewrites need a safety net

7.5 Target response budget

What a rewritten super-admin dashboard should cost. Use this as the acceptance criterion for the work above.

Metric	Today (est.)	Target	How
Database queries per request	~134	8–12	Aggregates + GROUP BY + batching
Sequential round-trips	~134	3–4	Promise.all in groups
Rows transferred	Entire tables	~20	SUM in SQL, not JavaScript
Full table scans	All of them	0	Section 6 indexes
Response time, cold	3–12 s	< 500 ms	All of the above
Response time, cached	3–12 s	< 30 ms	60-second response cache
Concurrent dashboards before pool exhaustion	~20	200+	Connections held for ms, not seconds

Estimates throughout are analytical, derived from counted queries and stated latency assumptions. The Day 1 measurement work exists precisely so that the second time you read this table, both columns are real numbers.

Appendix — Evidence Index

Line numbers are as of the audit date; verify against your working copy before editing.

ID	File	Location
Q-1	migrations/20221201092123-add_index_in_tables.js	the only addIndex migration; 30 tables, all id-led
Q-2	app/library/common.js	getTotalStockByUser
Q-3	app/library/common.js	getTotalStockPriceByUser
Q-4	app/library/common.js	getOwnUserSaleProducts
Q-5	app/controllers/superadmin/dashboard.controller.js	912–1053 (while month < 13)
Q-6	app/library/common.js	avlStockUserIdsNew
Q-7	app/library/common.js	getSuperAdminId
P-1	app/controllers/superadmin/dashboard.controller.js	63–1153; 167 awaits in index
P-2	config/config.js	createMySqlConfig — external host, all environments
P-4	app/controllers/superadmin/dashboard.controller.js	63–1153; implicit globals at 85–126
P-5	common.js / dashboard.controller.js	70 + 25 console.log / compactLog calls
P-6	server.js	bodyParser.json({ limit: '50mb' }); no compression
P-7	app/controllers/superadmin/dashboard.controller.js	1155–1886 (indexNew, unrouted)
P-8	config/config.js	pool: { max: 20, min: 0, acquire: 60000 }
O-1	app/middlewares/requestLogger.js	duration computed before next()
O-2	config/config.js	logging: false
S-1	config/config.js	hardcoded credential fallbacks, all environments
—	app/routes/superadmin.routes.js	149–154 — the dashboard route

Counting method

Query counts were obtained by reading the super-admin branch of exports.index (lines 63–345) and the shared section (827–1153), then expanding each helper call by reading the helper in app/library/common.js and counting its own awaits, including the queries Sequelize issues for separate: true associations. The month loop was counted as iterations × awaits per iteration. Where a helper calls another helper, the inner count is included.

End of report.